

Greedy Method

- Greedy algorithm obtains an **optimal** solution by making a **sequence** of decisions.
- Decisions are made **one by one** in some order.
- Each decision is made using a **greedy-choice property** or greedy criterion.
- A decision, once made, is (usually) **not changed** later.

- It gives an optimal solution when applied to problems with the **greedy-decision** property.
- A **feasible solution** is a solution that satisfies the constraints.
- An **optimal solution** is a feasible solution that optimizes the **objective** function.

Greedy method control abstraction/ general method

Algorithm Greedy(a,n)

// a[1:n] contains the n inputs

{

 solution= //Initialize solution

for i=1 to n **do**

 {

 x:=Select(a);

if Feasible(solution,x) **then**

 solution=Union(solution,x)

 }

return solution;

}

Example: Largest k-out-of-n Sum

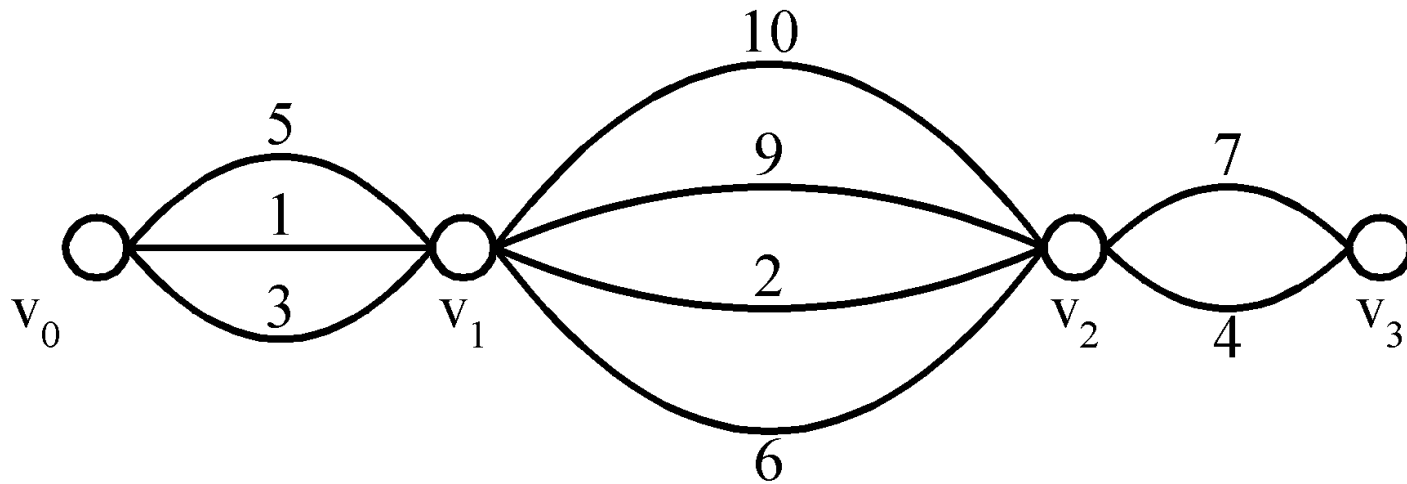
- Problem
 - Pick k numbers out of n numbers such that the **sum** of these k numbers is the **largest**.
- Exhaustive solution
 - There are C_k^n choices.
 - Choose the one with subset sum being the largest
- Greedy Solution *Is the greedy solution **always optimal**?*
 - FOR $i = 1$ to k
 - pick out the **largest** number and
 - delete** this number from the
 - input.
 - ENDFOR



Example:

Shortest Path on a Special Graph

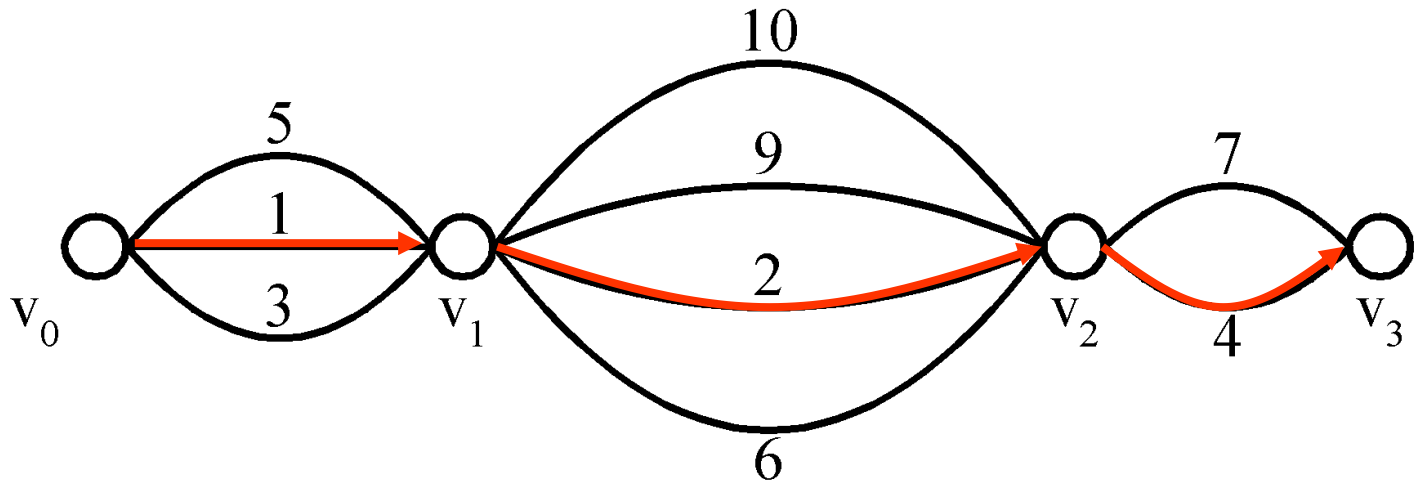
- Problem
 - Find a **shortest** path from v_0 to v_3
- Greedy Solution



Example:

Shortest Paths on a Special Graph

- Problem
 - Find a **shortest** path from v_0 to v_3
- Greedy Solution *Is the solution **optimal**?*

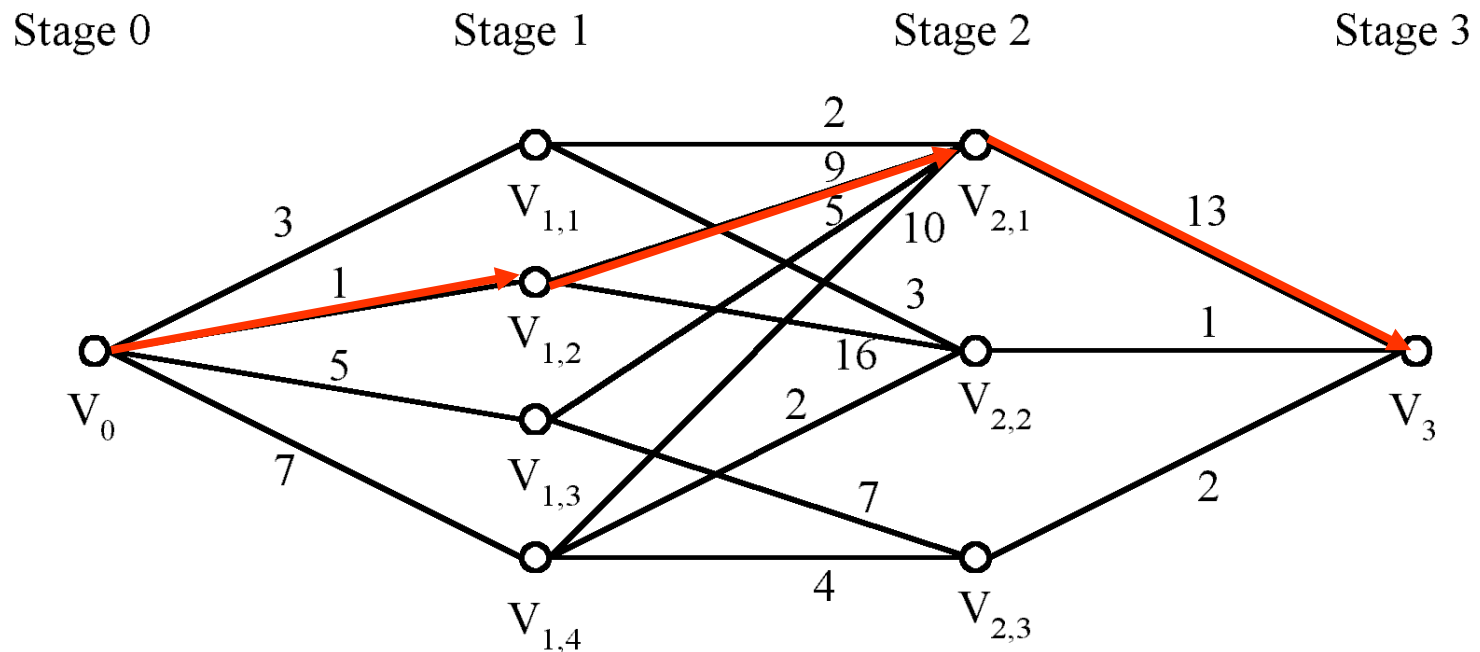


Example:

Shortest Paths on a Multi-stage Graph

*Is the greedy solution **optimal**?*

- Problem
 - Find a **shortest** path from v_0 to v_3



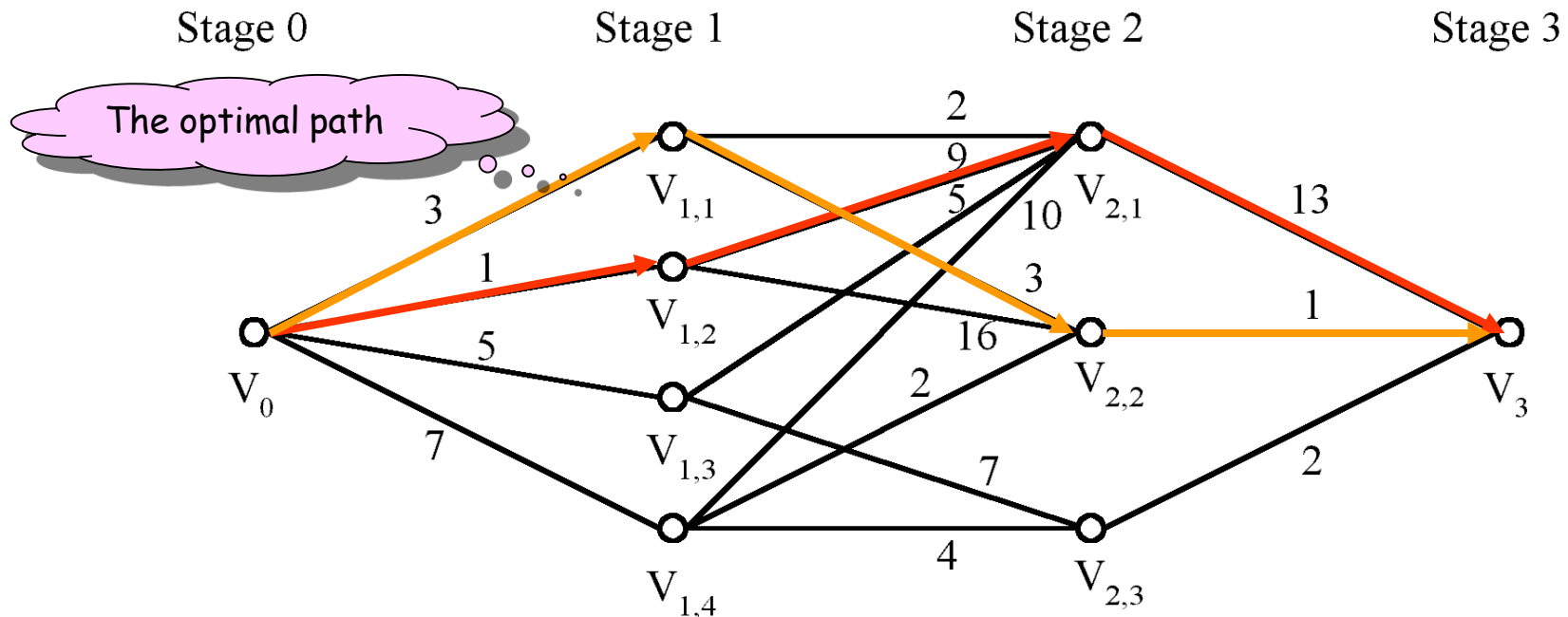
Example:

Shortest Paths on a Multi-stage Graph

Is the greedy solution optimal? ✖

- Problem

- Find a **shortest** path from v_0 to v_3



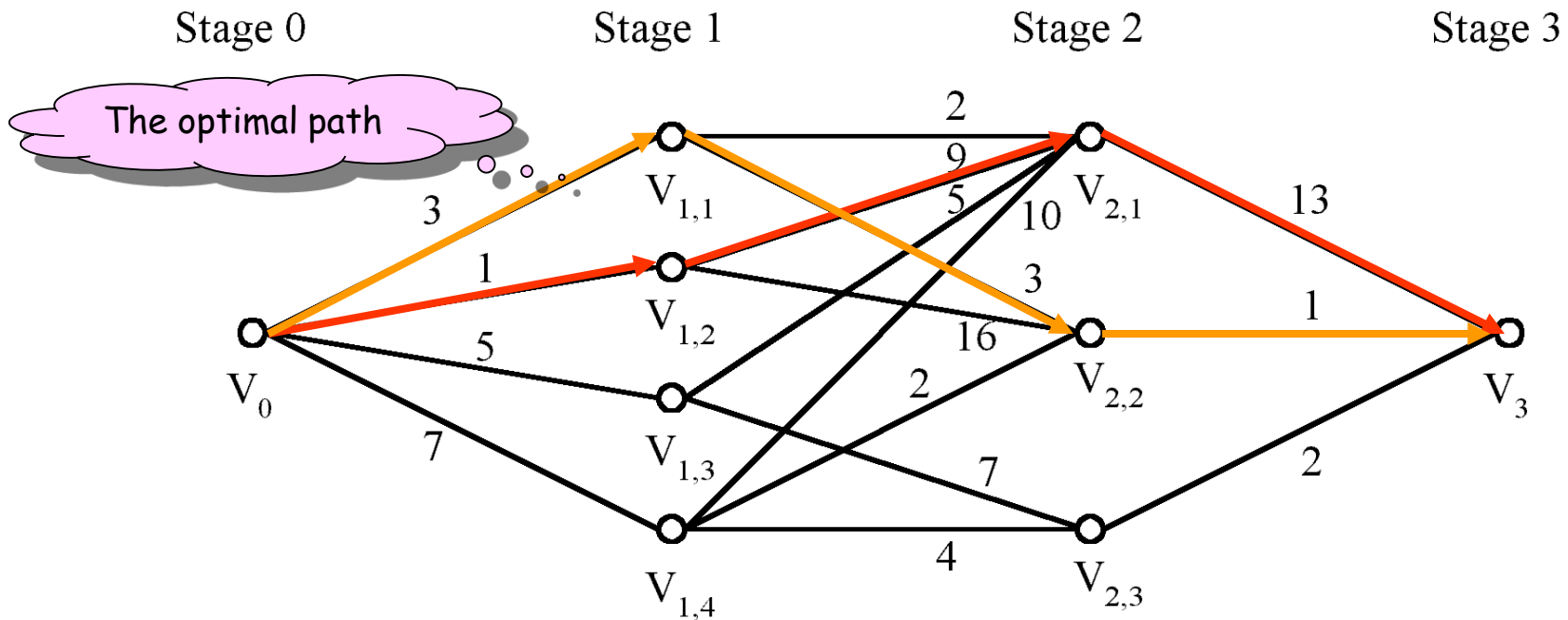
Example:

Shortest Paths on a Multi-stage Graph

Is the greedy solution optimal?

What algorithm can be used to find the optimum?

Find a shortest path from v_0 to v_3



The Fractional Knapsack Problem

- Given: A set S of n items, with each item i having
 - p_i - a positive profit
 - w_i - a positive weight
- Goal: Choose items, allowing fractional amounts(x_i), to maximize total profit but with weight at most m .

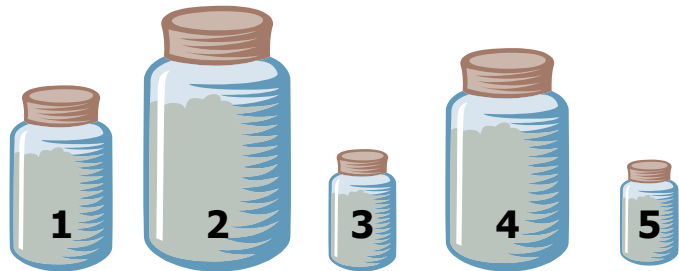
$$\begin{aligned} &\text{maximize } \sum_{1 \leq i \leq n} p_i x_i \\ &\text{subjected to } \sum_{1 \leq i \leq n} w_i x_i \leq m \\ &\quad \text{and } 0 \leq x_i \leq 1, \quad 1 \leq i \leq n \end{aligned}$$

The Fractional Knapsack Problem

Greedy decision property:-

Select items in **decreasing** order of **profit/weight**.

Items:



w_i : 4 ml 8 ml 2 ml 6 ml 1 ml

p_i : \$12 \$32 \$40 \$30 \$50

Value:
(\$ per ml) 3 4 20 5 50



10 ml

"knapsack"

Solution:

- 1 ml of i_5
- 2 ml of i_3
- 6 ml of i_4
- 1 ml of i_2

- Solution vector

$$(x_1, x_2, x_3, x_4, x_5) = (0, 1/8, 1, 1, 1)$$

- Profit $= 12*0 + 32*1/8 + 40*1 + 30*1 + 50*1$
 $= 0+4+40+30+50$
 $= 124.$

Greedy algorithm for the fractional Knapsack problem

Algorithm GreedyKnapsack(m,n)

//P[1:n] and w[1:n] contain the profits and weights

// respectively of the n objects ordered such that

// $p[i]/w[i] \geq p[i+1]/w[i+1]$.

//m is the knapsack size and x[1:n] is the solution

// Vector.

```
{
    for i=1 to n do x[i]=0; // Initialize x.
    U=m;
    for i=1 to n do
    {
        if ( w[i]>U ) then break;
        if x[i]=1; U=U-w[i];
    }
    if ( i <=n) then x[i]= U/w[i];
}
```

If you do not consider the time to sort the items, then the time taken by the above algorithm is **$O(n)$** .

0/1 Knapsack Problem

- An item is either included or not included into the knapsack.

Formally the problem can be stated as

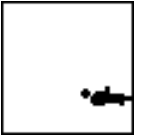
$$\text{maximize } \sum_{1 \leq i \leq n} p_i x_i$$

$$1 \leq i \leq n$$

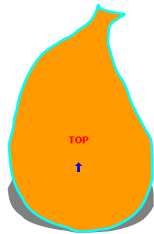
$$\text{subjected to } \sum_{1 \leq i \leq n} w_i x_i \leq m$$

$$1 \leq i \leq n$$

$$\text{and } x_i = 0 \text{ or } 1, \quad 1 \leq i \leq n$$



Which items should be chosen to **maximize** the amount of **money** while still keeping the overall **weight under m kg** ?

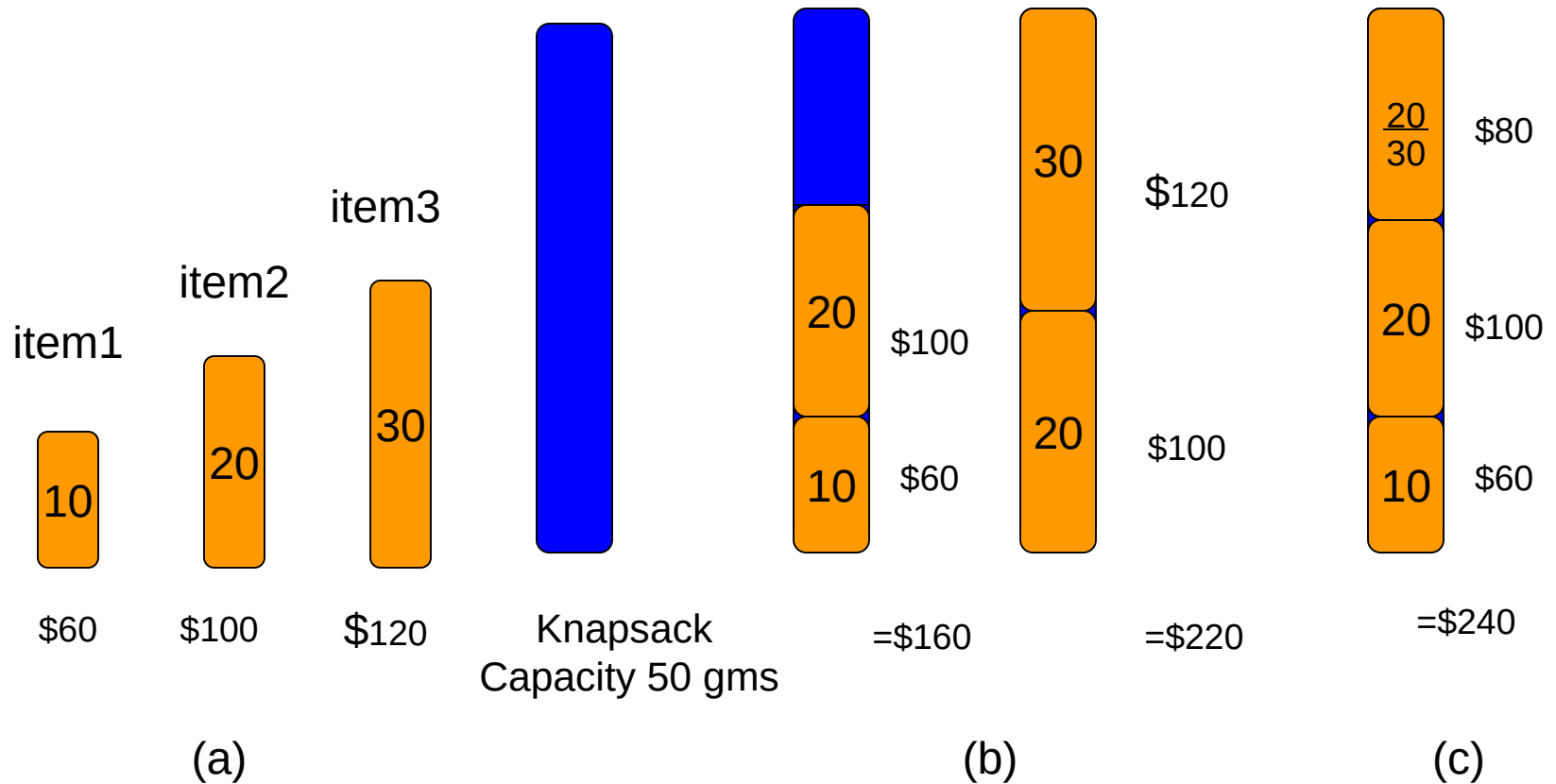


m



Is the fractional knapsack algorithm applicable?

- The greedy method works for **fractional** knapsack problem, but it does not for **0/1** knapsack problem.
- Ex:-

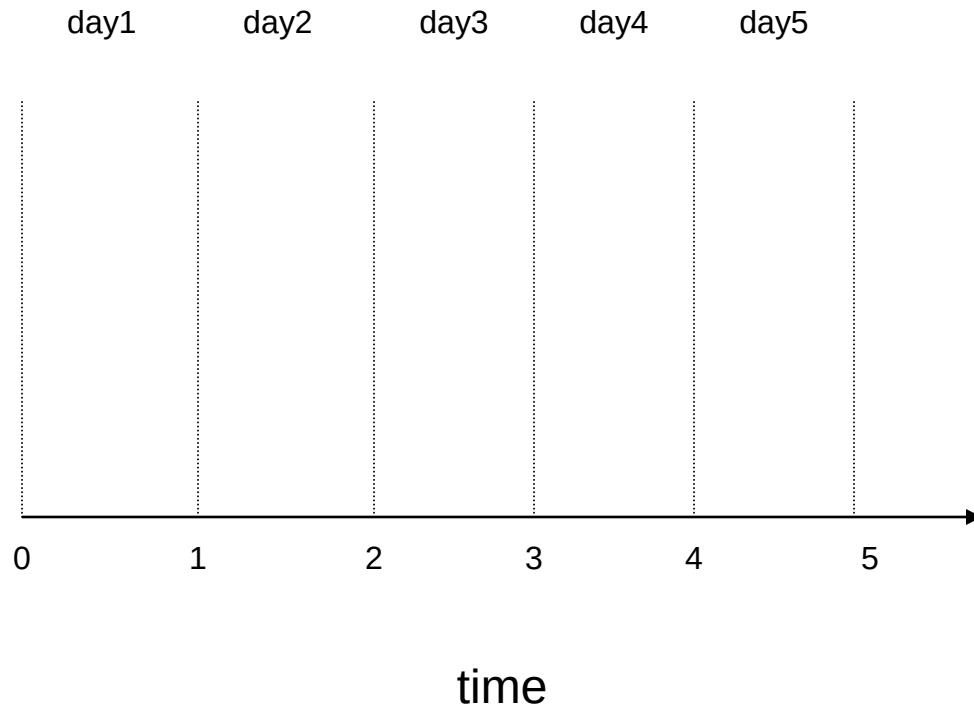


- There are 3 items, the knapsack can hold 50 gms.
- The value per gram of item 1 is 6, which is greater than the value per gram of either item2 or item3.
- The *greedy approach* (Decreasing order of profit's/weight's), does not give an optimal solution.
- As we can see from the above fig., the *optimal solution* takes item2 and item3.
- For the *fractional* problem, the *greedy approach* (Decreasing order of profit's/weight's) gives an *optimal solution* as shown in fig c.

Job sequencing with deadlines

- We are given a set of n jobs.
- *Deadline* $d_i \geq 0$ and a *profit* $p_i > 0$ are associated with each *job* i .
- For any job profit is earned if and only if the job is *completed by its deadline*.
- To complete a job, a job has to be processed by a machine for *one unit* of time.
- Only *one machine* is available for processing jobs.
- A *feasible solution* of this problem is a *subset of jobs* such that each job in this subset can be completed by its deadline
- The *optimal solution* is a feasible solution which will *maximize* the total profit.
- The *objective* is to find an *order of processing of jobs* which will *maximize the total profit*.

Ex:- $n=4$, $(p_1, p_2, p_3, p_4) = (100, 10, 15, 27)$
 $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$



Ex:-n=4, (p1,p2,p3,p4)=(100,10,15,27)
(d1,d2,d3,d4)=(2,1,2,1)

The maximum deadline is 2 units, hence the feasible solution set must have ≤ 2 jobs.

	feasible solution	processing sequence	value
1.	(1,2)	2,1	110
2.	(1,3)	1,3 or 3,1	115
3.	(1,4)	4,1	127
4.	(2,3)	2,3	25
5.	(3,4)	4,3	42
6.	(1)	1	100
7.	(2)	2	10
8.	(3)	3	15
9.	(4)	4	27

Solution 3 is optimal.

Greedy Algorithm for job sequencing with deadlines

1. Sort p_i into decreasing order. After sorting $p_1 \geq p_2 \geq p_3 \geq \dots \geq p_i$.
2. Add the next job i to the solution set if i can be completed by its deadline. Assign i to time slot $(r-1, r)$, where r is the largest integer such that $1 \leq r \leq d_i$ and $(r-1, r)$ is free.
3. Stop if all jobs are examined. Otherwise, go to step 2.

Ex:- 1) $n=5$, $(p_1, \dots, p_5) = (20, 15, 10, 5, 1)$ and
 $(d_1, \dots, d_5) = (2, 2, 1, 3, 3)$

The optimal solution is $\{1, 2, 4\}$ with a profit of 40.

Ex:- 2) $n=7$, $(p_1, \dots, p_7) = (3, 5, 20, 18, 1, 6, 30)$ and
 $(d_1, \dots, d_7) = (1, 3, 4, 3, 2, 1, 2)$

Find out an optimal solution.

Algorithm JS(d,j,n)

//d[i]≥1, $1 \leq i \leq n$ are the deadlines.

//The jobs are ordered such that $p[1] \geq p[2] \dots \geq p[n]$

// j[i] is the i^{th} job in the optimal solution, $1 \leq i$

{

 d[0]=j[0]=0; // Initialize

 j[1]=1; // Include job 1

 k=1;

```
for i=2 to n do
```

```
{    //Consider jobs in Descending order of p[i].  
    // Find position for i and check feasibility of  
    // insertion.
```

```
    r=k;
```

```
    while( ( d[j[r]]> d[i] and ( d[j[r]] ≠ r )) do  
        r= r-1;
```

```
    if( d[i] > r )) then
```

```
    {
```

```
        // Insert i into j[ ].
```

```
        for q=k to (r+1) step -1 do j[q+1] = j[q];
```

```
        j[r+1] :=i;
```

```
        k:=k+1;
```

```
    }
```

```
}
```

```
return k;
```

```
}
```

Time taken by this algorithm is $O(n^2)$

OPTIMAL STORAGE ON TAPES

- Given n programs $P_1, P_2, \dots, P_n$ of length $L_1, L_2, \dots, L_n$ respectively, store them on a tape of length L such that Mean Retrieval Time (MRT) is a minimum.
- The retrieval time of the j^{th} program is a summation of the length of first j programs on tape.
- Let T_j be the time to retrieve program P_j .
- The retrieval time of P_j is computed as,

$$T_j = \sum_{k=1}^j L_k$$

Length of k^{th} program

- Mean retrieval time of n programs is the average time required to retrieve any program.
- It is required to store programs in an order such that their **Mean Retrieval Time is minimum**. MRT is computed as

The diagram illustrates the formula for Mean Retrieval Time (MRT) with annotations for its components:

$$\text{MRT} = \frac{1}{n} \sum_{i=1}^N T_j = \left(\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^i (L_k) \right)$$

Annotations in the diagram:

- An arrow points from the text "Average retrieval time over n programs" to the entire formula.
- An arrow points from the text "Time to retrieve jth program P_j" to the term T_j in the first part of the equation.
- An arrow points from the text "Length of kth program" to the term (L_k) inside the nested summation.

Optimal storage on tape is minimization problem which

$$\text{Minimize } \sum_{i=1}^n \sum_{k=1}^i L_k$$

Subjected to $\sum_{i=1}^n L_i \leq L$

Length of i^{th} program $\uparrow$

Length of tape $\nwarrow$

Storage on Single Tape

Algorithm Algorithm MRT_SINGLE_TAPE(L)

// Description : Find storage order of n programs to such that mean retrieval time is minimum

// Input : L is array of program length sorted in ascending order

// Output : Minimum Mean Retrieval Time

$T_j \leftarrow 0$

for i $\leftarrow$ 1 to n do

 for j $\leftarrow$ 1 to i do

$T_j \leftarrow T_j + L[j]$

 end

end

$MRT \leftarrow T_j / n$

Time Complexity analysis of Optimal Storage on Tapes

Primitive operation in above algorithm is the addition of program length, which is enclosed within two loops. The running time of algorithm is given by,

$$\begin{aligned} T(n) &= \sum_{i=1}^n \sum_{j=1}^i \theta(1) \\ &\quad \begin{array}{cc} \nearrow & \nwarrow \\ \text{Loop over } n & \text{Retrieval time for} \\ \text{programs} & j^{\text{th}} \text{ program} \end{array} \\ &= \sum_{i=1}^n \sum_{j=1}^i 1 \\ &= \sum_{i=1}^n i = 1 + 2 + 3 + \dots + n \\ &= \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2} \end{aligned}$$

$$T(n) = O(n^2)$$

Ex-1: Consider three programs (P1, P2, P3) with a length of (L1, L2, L3) = (5, 10, 3). Store them on a single tape.

Sol: Given Programs lengths 5, 10, 3

Sorted order lengths 3, 5, 10

storing optimally in the order P3, P1, P2

$$\begin{aligned}\text{Mean Retrieval Time (MRT)} &= 3 + (3+5) + (3+5+10)/3 \\ &= (3+8+18)/3 \\ &= 29/3 \\ &= 9.66\end{aligned}$$

Ex-2:

Given the program lengths $L = \{12, 34, 56, 73, 24, 11, 34, 56, 78, 91, 34, 91, 45\}$. Store them on three tapes and minimize MRT.

Sol:

Given data :

P _i	P ₁	P ₂	P ₃	P ₄	P ₅	P ₆	P ₇	P ₈	P ₉	P ₁₀	P ₁₁	P ₁₂	P ₁₃
L _i	12	34	56	73	24	11	34	56	78	91	34	91	45

First sort the programs in increasing order of their size.

Sorted data:

P _i	P ₆	P ₁	P ₅	P ₂	P ₇	P ₁₁	P ₁₃	P ₃	P ₈	P ₄	P ₉	P ₁₀	P ₁₂
L _i	11	12	24	34	34	34	45	56	56	73	78	91	91

Now distribute the files among all three tapes.

Tape 1	P ₆	P ₂	P ₁₃	P ₄	P ₁₂
Tape 2	P ₁	P ₇	P ₃	P ₉	
Tape 3	P ₅	P ₁₁	P ₈	P ₁₀	

$$\begin{aligned}
 \text{MRT}_{\text{Tape1}} &= ((11) + (11 + 34) + (11 + 34 + 45) + ((11 + 34 + 45 + 73)) + (11 + 34 + 45 + 73 + 91)) / 4 \\
 &= 563 / 5 \\
 &= 112.6
 \end{aligned}$$

$$\begin{aligned}
 \text{MRT}_{\text{Tape2}} &= ((12) + (12 + 34) + (12 + 34 + 56) + (12 + 34 + 56 + 78)) / 4 \\
 &= 340 / 4 \\
 &= 85
 \end{aligned}$$

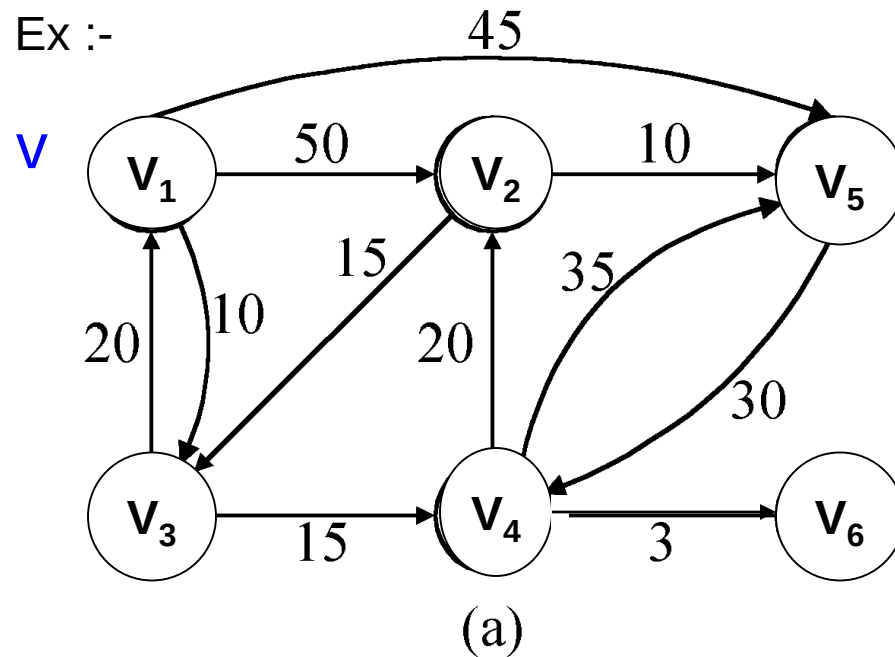
$$\begin{aligned}
 \text{MRT}_{\text{Tape3}} &= ((24) + (24 + 34) + (24 + 34 + 56) + (24 + 34 + 56 + 91)) / 4 \\
 &= 353 / 4 \\
 &= 88.25
 \end{aligned}$$

$$\begin{aligned}
 \text{MRT} &= (\text{MRT}_{\text{Tape3}} + \text{MRT}_{\text{Tape3}} + \text{MRT}_{\text{Tape3}}) / 3 \\
 &= (112.6 + 85 + 88.25) / 3 \\
 &= 95.28
 \end{aligned}$$

The Single-Source Shortest path Problem (SSSP)

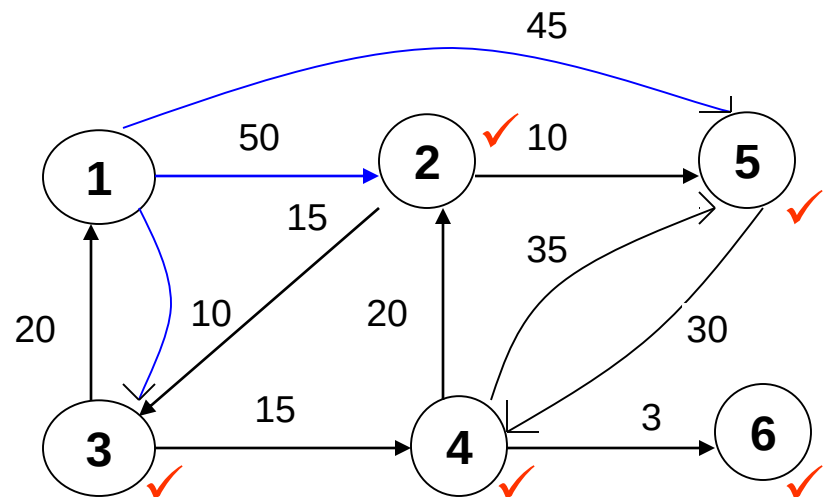
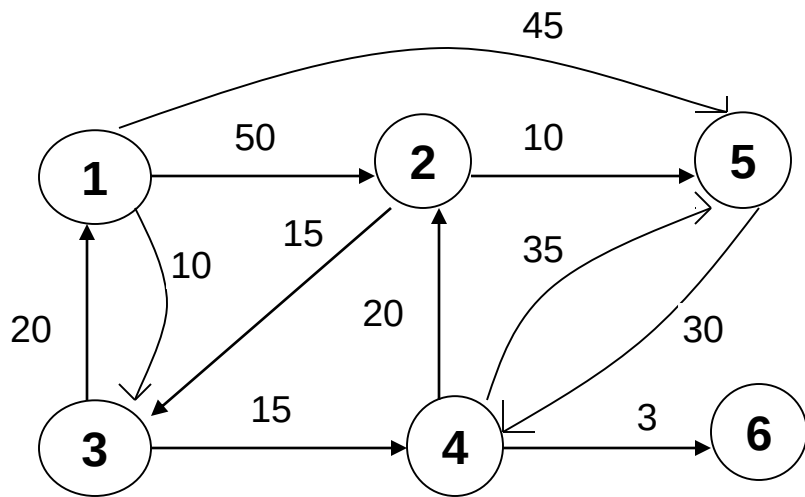
- Given a **positively** weighted directed graph G with a source vertex **v**, find the shortest paths **from v to all other vertices** in the graph.

Ex :-



	<u>Path</u>	<u>Length</u>
1)	v_1v_3	10
2)	$v_1v_3v_4$	25
3)	$v_1v_3v_4v_2$	45
4)	v_1v_5	45
5)	$v_1v_3v_4v_6$	28

(b)



Iteration	S	Dist[2]	Dist[3]	Dist[4]	Dist[5]	Dist[6]
Initial	{1}	50	10 ✓	∞	45	∞
1	{1,3}	50	10	25 ✓	45	∞
2	{1,3,4}	45	10	25	45	28 ✓
3	{1,3,4,6}	45	10	25	45 ✓	28
4	{1,3,4,5,6}	45 ✓	10	25	45	28

SSSP-Dijkstra's algorithm

- Dijkstra's algorithm assumes that $\text{cost}(e) \geq 0$ for each e in the graph.
- Maintains a set S of vertices whose SP from v (source) has been determined.
- a) Select the next minimum distance node u , which is not in S .
- (b) *for* each node w adjacent to u *do*

– // Relaxation

if($\text{dist}[w] > \text{dist}[u] + \text{cost}[u, w]$) then
 $\text{dist}[w] := \text{dist}[u] + \text{cost}[u, w]$;

- Repeat step (a) and (b) until $S = n$ (number of vertices).

1 Algorithm ShortestPaths($v, \text{cost}, \text{dist}, n$)

2 // $\text{dist}[j]$, $1 \leq j \leq n$, is the length of the shortest path

3 // from vertex v to vertex j in a digraph G with n
vertices.

4 // $\text{dist}[v]$ is set to zero. G is represented by its cost
adjacency

5 // matrix $\text{cost}[1:n, 1:n]$.

6 {

7 for $i := 1$ to n do

8 { // Initialize S .

9 $s[i] := \text{false}$; $\text{dist}[i] := \text{cost}[v, i]$;

10 }

11 $s[v] := \text{true}$; // put v in S .

```
12  for num:=2 to n do
13      {
14          Determine n-1 paths from v.
15          Choose u from among those vertices not in S
            such that dist[u] is minimum;

17          s[u]:=true; // Put u in S.
18          for ( each w adjacent to u with s[w]= false) do
19              // Uupdate distance
20              if( dist[w]>dist[u]+cost[u,w]) ) then
21                  dist[w]:=dist[u]+cost[u,w];
22      }
23 }
```

Time complexity of Dijkstra's Algorithm

- The for loop of line 7 takes $o(n)$.
- The for loop of line 12 takes $o(n)$.
 - Each execution of this loop requires $o(n)$ time at lines 15 and 18.
 - So the total time for this loop is $o(n^2)$.
- Therefore, total time taken by this algorithm is $o(n^2)$.

OPTIMAL MERGE PATTERNS

Merge a set of sorted files of different length into a single sorted file. We need to find an optimal solution, where the resultant file will be generated in minimum time.

If the number of sorted files are given, there are many ways to merge them into a single sorted file.

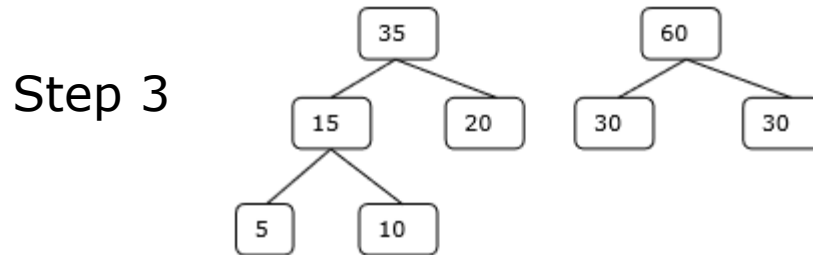
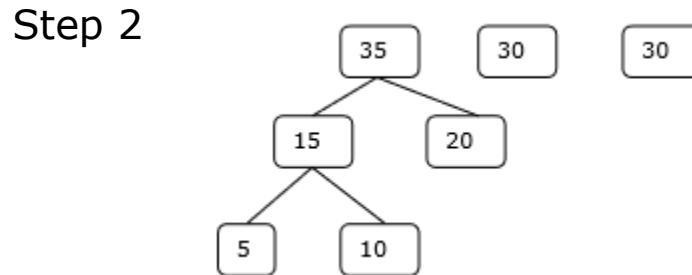
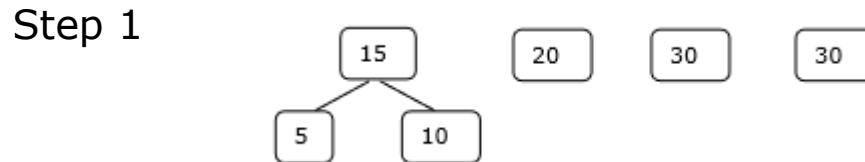
This merge can be performed **pair wise**. Hence, this type of merging is called as **2-way merge patterns**.

- we want to determine an **optimal** way of merging many files together. At each step, two shortest sequences are merged.
- To merge a **p-record file** and a **q-record file** requires possibly $p + q$ record moves, the obvious choice being, merge the two smallest files together at each step.
- Two-way merge patterns can be represented by binary merge trees. Let us consider a set of n sorted files $\{f_1, f_2, f_3, \dots, f_n\}$. Initially, each element of this is considered as a single node binary tree.

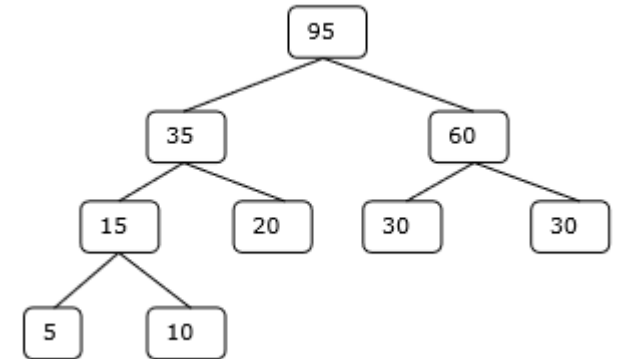
Pseudo code

```
for i := 1 to n - 1 do
  declare new node
  node.leftchild := least (list)
  node.rightchild := least (list)
  node.weight := ((node.leftchild).weight) +
    ((node.rightchild).weight)
  insert (list, node);
return least (list);
```

Let us consider the given files, f_1, f_2, f_3, f_4 and f_5 with 20, 30, 10, 5 and 30 number of elements respectively.



Step 4



Hence, the solution takes $15 + 35 + 60 + 95 = 205$ number of comparisons.

C Program

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int n, files[] ; // Read n , files[] from the user with scanf()
    int minCost = optimalMerge(files, n);
    printf("Minimum cost of merging is: %d Comparisons\n", minCost);
    return 0;
}

int optimalMerge(int files[], int n)
{
    // Sort the files in ascending order
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (files[j] > files[j + 1]) {
                int temp = files[j];
                files[j] = files[j + 1];
                files[j + 1] = temp;
            }
        }
    }
    int cost = 0;
```

```
while (n > 1) {  
    // Merge the smallest two files  
    int mergedFileSize = files[0] + files[1];  
    cost += mergedFileSize;  
    // Replace the first file with the merged file size  
    files[0] = mergedFileSize;  
    // Shift the remaining files to the left  
    for (int i = 1; i < n - 1; i++) {  
        files[i] = files[i + 1];  
    }  
    n--; // Reduce the number of files  
    // Sort the files again  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - i - 1; j++) {  
            if (files[j] > files[j + 1]) {  
                int temp = files[j];  
                files[j] = files[j + 1];  
                files[j + 1] = temp;  
            }  
        }  
    }  
}  
return cost;  
}
```